```
  def test_two_words_with_adjacent_repeat
    assert_equal({"cat" => 2, "sat" => 1}, count_frequency(["cat", "cat", "sat"]))
  end

  def test_two_words_with_non_adjacent_repeat
    assert_equal({"cat" => 2, "sat" => 1}, count_frequency(["cat", "sat", "cat"]))
  end
end
```

produces:

```
Run options: --seed 61320
# Running:

.....
Finished in 0.000472s, 10593.2203 runs/s, 10593.2203 assertions/s.
5 runs, 5 assertions, 0 failures, 0 errors, 0 skips
```

In previous versions of the book, we stopped here. However, since then, the Ruby Standard Library has evolved, and the Array class now has a tally method that does exactly what our count_frequency method does. We can use tally instead:

tut_containers/word_freq/better_word_count.rb

```
require_relative "words_from_string"

raw_text = "The problem breaks down into two parts. First, given some text
as a string, return a list of words. That sounds like an array. Then, build
a count for each distinct word. That sounds like a use for a hash---we can
index it with the word and use the corresponding entry to keep a count."

word_list = words_from_string(raw_text)
counts = word_list.tally
sorted = counts.sort_by { |word, count| count }
top_five = sorted.last(5)

top_five.reverse_each do |word, count|
  puts "#{word}:  #{count}"
end
```

produces:

```
a:  6
the:  3
count:  2
word:  2
sounds:  2
```

And we get the same answer.

Blocks and Enumeration

In our program that wrote out the results of our word frequency analysis, we had the following loop:

```
top_five.reverse_each do |word, count|
  puts "#{word}:  #{count}"
end
```

The method reverse_each is an example of an *iterator*—a general term for a method that invokes a block of code repeatedly. Ruby also uses the term *enumerator* for such a method.